

MIP_PROMPT_LIBRARY_V2_PART3_METADATA.md

Mainframe Intelligence Platform

Enterprise Prompt Library V2

Part 3 - Metadata Prompts

Version: 2.0

Status: Approved

Purpose

Metadata is the canonical representation of a legacy system.

Source code is implementation.

Metadata is knowledge.

MIP transforms:

Source Code

↓

Metadata

↓

Relationships

↓

Knowledge Graph

↓

Reasoning

↓

Modernization Intelligence

Without metadata, every modernization effort becomes manual and expensive.

Metadata Design Principles

Evidence Based

Every metadata element must reference source evidence.

Canonical

One representation regardless of source language.

Traceable

Every entity must be traceable back to source artifacts.

Graph Ready

Metadata must be designed for graph traversal.

Modernization Ready

Metadata should support future migration decisions.

Prompt 11

Program Metadata Model

File

prompts/metadata/program_entity.md

Purpose

Design the canonical Program entity used throughout MIP.

Context

Programs are primary execution units.

The Program entity becomes the most connected node in the platform.

Inputs

COBOL Metadata

Instructions

Design a canonical Program entity.

Include:

Identity

- Program Name
- Program Type
- Source Location

Dependencies

- Called Programs
- Copybooks
- Tables
- Files

Execution

- Entry Points
- Batch Usage
- Online Usage

Metrics

- Lines of Code
- Complexity
- Dependency Count

Generate:

- Pydantic Model
- SQLite Mapping
- Graph Mapping

Expected Output

Program Entity Specification

Validation Rules

Versioning Strategy

Constraints

Must support millions of relationships.

Success Criteria

Program entity supports graph and modernization use cases.

Prompt 12

Job Metadata Model

File

prompts/metadata/job_entity.md

Purpose

Create canonical representation of batch jobs.

Context

Batch jobs are often true system entry points.

Inputs

JCL Metadata

Instructions

Design Job entity.

Include:

- Job Name
- Steps
- Programs
- Procedures
- Datasets

Generate:

- Metadata Model
- Graph Representation
- Lineage Mapping

Expected Output

Job Entity Definition

Relationship Model

Validation Rules

Constraints

Support large enterprise batch ecosystems.

Success Criteria

Job execution flows become graph-ready.

Prompt 13

Dataset Metadata Model

File

prompts/metadata/dataset_entity.md

Purpose

Represent enterprise datasets consistently.

Context

Datasets connect jobs, programs, and business processes.

Inputs

JCL Metadata COBOL Metadata

Instructions

Model:

- Dataset
- Dataset Type
- Producers
- Consumers
- Retention
- Ownership

Generate lineage-ready metadata.

Expected Output

Dataset Entity

Lineage Relationships

Governance Metadata

Success Criteria

Data lineage can be reconstructed.

Prompt 14

DB2 Metadata Model

File

prompts/metadata/db2_entity.md

Purpose

Represent database dependencies consistently.

Context

DB2 objects often become future microservice boundaries.

Inputs

DB2 Metadata

Instructions

Design metadata for:

- Tables
- Views
- Aliases
- Indexes

Capture:

- CRUD Usage
- Access Frequency
- Sharing Level

Expected Output

DB2 Entity Model

Relationship Definitions

Migration Metadata

Success Criteria

Database dependencies support modernization analysis.

Prompt 15

Relationship Metadata Model

File

prompts/metadata/relationship_model.md

Purpose

Create canonical relationship taxonomy.

Context

Relationships power the knowledge graph.

Inputs

All metadata entities.

Instructions

Design relationship types.

Core Types:

- CALLS
- EXECUTES
- USES
- READS
- WRITES
- CONTAINS
- DEPENDS_ON
- PRODUCES
- CONSUMES

Generate:

- Taxonomy
- Cardinality Rules
- Validation Rules

Expected Output

Relationship Catalog

Schema Definition

Graph Mapping

Success Criteria

Every dependency can be represented consistently.

Metadata Completion Criteria

MIP should be able to answer:

- What is a Program?
- What is a Job?
- What is a Dataset?
- What is a Table?
- How are they related?
- How do they execute together?
- What depends on what?

Only after metadata is complete should graph construction begin.

End of Part 3.